Frameworks

¿Que quiere decir reusar? ¿Qué es lo que reusamos?

Cuando programamos, rara vez se hacen todos los requerimientos desde 0, desde el analisis buscamos reutilizar cosas que ya sabemos que funcionan.

- Se reusan conceptos e ideas que funcionan.
- Diseños o estrategias de diseño (que imtamos, implementamos)
  - Por ejemplo, los patrones de diseño son una forma de reutilizar ya que cuando nos encontramos un problema particular, reutilizamos todo el diseño del patrón que ofrece una solución
- Funciones y estructuras de datos
  - Por ejemplo, colecciones, archivos, etc.
- Componentes y servicios (que invocamos)
  - Por ejemplo, si hay que desarollar una aplicación que en algún lugar debe mostrar un mapa con todos los contactos ubicados en dicho mapa, para ello, se puede utilizar el servicio de Mapa de Google y decirle directamente que indique cada contacto y obtemos un html que podemos utilizar en nuestra aplicación
- Aplicaciones completas que adaptamos e integramos
  - Por ejemplo, nos piden una aplicación y hay disponibles aplicaciones configurables adaptables que al agregarle, por ejemplo, pluggins, se convierte en la aplicación que necesitamos.

¿Por qué es importante pensar en reuso?

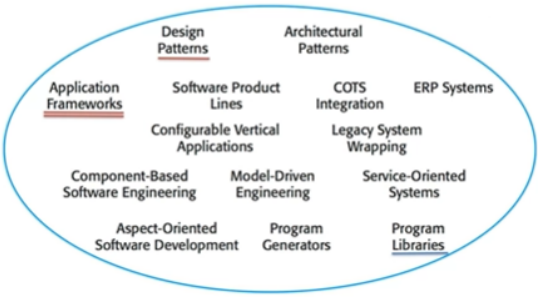
- Para aumentar la productividad, es decir, realizar más y mejores programas con menos tiempo y menos gente
  - Reducir los costos de desarrollo y mantenimiento
  - Reducir los tiempos de entrega y puesta en el mercado
  - Agilidad de mercado y customización en masa
- Para aumentar la calidad
  - El reúso aprovecha mejoras y corecciones, el tiempo de madurez y confiabilidad, es decir, al reutilizar se utilizan partes que ya están maduras, fueron mejoradas y tienen calidad
  - Se aprovecha el conocimiento de los especialistas (que queda encapsulado en componentes reusables), es decir, no hay que perder tiempo pensando como resolver un problema

¿Qué dificultades trae reutilizar?

- Por lo general se reutilizan cosas que no programamos nosotros, esto trae problemas de mantenimiento si no tenemos control de los componentes que reusamos.
  - Nuestro sistema queda “atado” a las decisiones que tomen los autores de los componentes que utilizamos
  - Por ejemplo, si todo nuestro sistema depende de una libreria externa y esta es rediseñada, cambiando nombres de funciones o lógica, podría implicar una sobreescritura del código ya implementado para adaptarlo nuevamente.
- Algunos programadores prefieren hacer todo ellos mismos ya que les genera más confianza y seguridad al escribir su código porque lo entienden al 100%. Sin embargo, no es escalable, se pierde tiempo resolviendo problemas que otros ya lo solucionaron mejor. Es más viable aprender a reutilizar.
- Hacer software reusable es más dificil y costoso
  - Hacer que un componente sea reutilizable implica que debe funcionar en muchos contextos, no solo el actual. Esto requiere mucha abstracción, manejo de errores y documentación, lo cual es más caro y dificil que hacer código para un solo uso específico.

- Encontrar, aprender a usar y adaptar componentes reusables requiere esfuerzo adicional.
    - Implica tiempo de investigación para encontrar el componente adecuado, aprender su API y configurarlo para que encaje en nuestro sistema.

¿Qué estrategias tenemos?



Se hace un enfoque en patrones de diseño, frameworks y librerias, principalmente para diferenciar las librerias de los frameworks y resaltar caracteristicas importantes de los frameworks.

Librerias de clases / Toolkits

Una librería es un **artefacto reutilizable de código** compuesto por clases, funciones u otros componentes que pueden ser utilizados por aplicaciones.

∴ Las **librerias de clases** son un conjunto de clases cohesivas (relacionadas entre sí) que se encuentran juntas y ofrecen soluciones a problemas comunes y generales a la mayoría de las aplicaciones (ej. estructuras de datos, manejos de archivos, utilidades matemáticas, etc.), sin estar enfocadas en un dominio específico.

- Cada clase de la librería resuelve un problema concreto y puede utilizarse de forma independiente del resto, no espera nada de nuestro código
- Cada clase de la librería es independiente del contexto de uso (no está pensada para una aplicación específica)
- No hay fuerte dependencia estructural entre las clases, por lo general, se pueden usar algunas sin necesidad de incorporar toda la librería.
    - Por ejemplo, podemos utilizar solo una clase especifíca de una librería que nos sirva para resolver una necesidad puntual, sin utilizar las demás.
- En el uso de una librería, nuestro código controla/usa a los objetos de las librerías, es decir, controla el flujo de ejecución y decide cúando y cómo invocar las funcionalidades que la librería ofrece (no hay inversión de control).

Ejemplos de librerías en Java

Java
Codec (apache common) Clases (hoy, 18) que implementan algoritmos de encoding/decoding algorithms (por ejemplo, phonetic, base64, URL).
Compress (apache common) Clases para trabajar con archivos tar, zip, bzip2, etc.
Math (apache common) Clases que implementan componentes autocontenidos de matemáticas y estadísticas.
java.útil.collections Clases que implementan estructuras de datos (colecciones) y algoritmos para manipularlas

Frameworks Orientados a Objetos

Observación: La materia lo especifica desde OO pero es lo mismo framework que framework orientado a objetos

¿Qué es un framework?

- Un framework puede entenderse como el resultado de aplicar refactoring y patrones de diseño para abstraer lo común a una familia de aplicaciones similares.
    - Un framework se puede ver como el resultado de refactorizar una aplicación, no solo para mejorar su calidad interna sino para abstraer lo que es común todas una familia de aplicaciones similares.
    - En ese proceso de refactoring se identifica lo que se mantendrá constante y será, a partir de ese momento, responsabilidad de los desarrolladores del framework y lo que podrá variar de aplicación a aplicación y será responsabilidad de los desarrolladores de aplicaciones que usen el framework.
- Los frameworks forman parte de las estrategias de "reuso"/"reutilización de software ya que capturan diseño y arquitectura reusable, además de implementación.
- Los frameworks son artefactos de software que cumplen como función principal capturar diseño e implementación para que sea reutilizable.
- Es una aplicación "semi completa" y "reusable", que puede ser especializada para construit aplicaciones concretas
- Un framework es un conjunto de clases concretas, abstractas e interfaces relacionadas que proveen una arquitectura y una solución reusable para una familia de aplicaciones/problemas relacionadas.
    - Por ejemplo, frameworks para hacer interfaces web, para hacer editores gráficos.
- Las clases en el framework se relacionan de manera que resuelven la mayor parte del problema, juntas conforman un todo.
- El código del framework controla al nuestro, controla el flujo principal de ejecución → "Inversión de control"
    - La mayor parte/diseño de nuestra aplicación la resuelve/implementa el framework, por lo tanto, la mayor parte del tiempo se ejecuta el código del framework que llama/invoca al código escrito por nosotros (los desarrolladores).
- Los desarolladores incorporan el framework en sus aplicaciones y lo especializan mediante puntos de extensión (hotspot) para cubrir sus necesidades
- El framework define el esqueleto general de la solución y el desarollador define los aspectos específicos (con sus propias caracteristicas) para completar el esqueleto.
- El framework puede verse como un subsistema de mi aplicación, casi completo, en el que toda la problematica que habia que resolver para construir ese subsistema ya esta resuelta y tiene ciertos huecos (puntos de extensión) que deben completarse para que ese subsistema semicompleto pueda integrarse dentro de mi aplicación final concreta.

Ejemplo: django

Carasteristicas interesantes de frameworks:

⇛ A diferencia de una libreria, un framework no solo ofrece un conjunto de funcionalidades/clases útiles si no que también provee una estructura coherente sobre la cual construir aplicaciones

⇛ Al igual que las librerias de clases, un framework son un conjunto de clases, interfaces y componentes relacionados; Sin embargo, estas clases están organizadas/juntas porque resuelven una gran parte de una problematica especifica de un dominio, y facilitan la construcción de un determinado tipo de aplicación o subsistema de aplicación.

> ⇛ Por ejemplo, un framework web puede incluir clases concretas, abstractas e interfaces que trabajan juntas para resolver aspectos comunes al desarrollo de aplicaciones web

⇛ Las clases de un framework están fuertemente relacionadas y diseñadas para trabajar en conjunto y resolver un problema.A diferencia de muchas librerías, no suele ser habitual utilizar clases del framework de forma completamente aislada, ya que forman parte de una arquitectura integrada.

Dos perspectivas sobre los frameworks (dos gorros diferentes)

- El gorro del que reusa el artefacto reutilizable (usuario de frameworks-reutilizar)
    - El desarrollador utiliza el framework como base y siente que está completando el diseño previamente definido por otros.

    - Su tarea principal es especializar o configurar los puntos de extensión (hotspots) para adaptar el framework a sus necesidades.
- El gorro del que diseña el framework (diseñador)
    - Implica construir una solución general (código) sabiendo que otros desarrolladores continuarán el trabajo el día de mañana.
    - El diseñador debe definir reglas claras, establecer la arquitectura base y crear puntos de extensión (ganchos) donde alguien luego pueda introducir variaciones.
    - Debe decidir qué aspectos serán constantes para todas las aplicaciones hechas con el framework (frozenspots) y cuáles podrán variar (hotspots).

Clasificación de frameworks

Frameworks de infraestructura.

– Aquellos que resuelven problemas comunes a cualquier tipo de programa, problemas generales del desarrollo de software (ej. concurrencia, logging, interfaces gráficas basicas, etc.) que, por lo general, no son percibidos completamente por el usuario de la aplicación, es decir, resuelven problemas más tecnicos de los programadores no funcionalidades visibles para los usuarios finales.

– Ofrecen una instraestructura portable y eficiente sobre la cual construir una gran variedad de aplicaciones.

Frameworks de integración

– Aquellos cuyo objetivo es actuar como interfaz o intermediario entre distintas partes (sistemas o componente).

– Se utilizan para integrar aplicaciónes distribuidas (donde los componentes se ejecutan en múltiples computadoras y operan de manera independiente), facilitando la comunicación y coordinación entre ellas

– Estan diseñados para aumentar la capacidad de los desarrolladores de modularizar, reusar y extender su infraestructura de software permitiendo que distintos componentes trabajen juntos de forma transparente en entornos distribuidos.

– Su función principal es conectar mundos diferentes, por ejemplo, el mundo de la base de datos relacionales con el mundo de los objetos.

– Se puede decir que son un caso especial de frameworks de infraestructura, ya que resuelven un problema técnico (la integración) en lugar de un problema de negocio directo.

– Ejemplos: Object Relational Mapping, Message Oriented Middleware, etc.

Frameworks de aplicación

– Aquellos que capturan diseño e implementación especificos para resolver problemas dentro de un dominio particular.

– Abordan problemas de dominio importantes para las empresas, basados en necesidades reales de los usuarios, lo que hace que su desarrollo tenga un alto retorno de inversión.

– Son los que capturan el conocimiento de una gran familia de clientes, permitiendole a una empresa ser más productiva y tener software de calidad.

– Son la base para la implementación de lineas de productos (SPL)

Por ejemplo: Se hace una aplicación para una flota de taxis, se vende a un lugar especifico, lo conveniente sería querer vender la misma aplicación a otras flotas de taxis, cambiando pequeños detalles. Si se encapsula todo lo compartido en un framework se puede lograr un framework especializado para aplicaciones de flota de taxi.

Basicamente, permite obtener un framework para sistemas de gestión de flotas de taxis que permite reutilizar funcionalidades comunes y adaptar variaciones específicas para distintos clientes.

Algunos frameworks conocidos...

Java
Hibernate Framework de integración que resuelve el mapeo de objetos a bases de datos relacionales
jBPM Framework de aplicación (y suite de herramientas) para construir aplicaciones en las que se modelan, ejecutan, y monitorean procesos de negocios.
jUnit Framework de aplicación/infraestructura que resuelve la automatización de tests de unidad
libGDX Framework de aplicación, para construir juegos en Java, multiplataforma.
Spring Familia de frameworks de infraestructura e integración, enfocando una amplia variedad de necesidades (Testing, Data, Web, etc.)

Frozenspot vs Hotspots

» Cuando se diseña un framework, hay que decidir que es lo que va a ser igual en todas las aplicaciones (frozenspot) y que parte sera la variable (hotspot). *Hotspot es lo opuesto a frozenspot*

Estos dos conceptos se desconectan del código, no es que uno se pregunta ¿esta clase es un hotspot? ¿esta clase es un frozenspot?, eso no tiene sentido.

Estos dos conceptos se describen más al nivel de especificación de requerimientos.

¿Qué es el frozenspot?

⁘ Son las partes del framework que serán igual para todos los artefactos a construir con él.

⁘ Son aquellos aspectos comunes de todas las aplicaciones hechas con el framework que no se pueden cambiar (no modificables) definidos por el diseñador del framework.

⁘ Todas las aplicaciones construídas con un mismo framework comparten aspectos en común ya que forman parte del diseño fijo (arquitectura base) del framework.

¿Qué es un hotspot?

⁘ Son las partes del artefacto a construir con el framework que puedo cambiar.

⁘ Son los puntos de variabilidad del framework, es decir, aquellos lugares donde el comportamiento puede cambiar o especializarse.

⁘ Son aquellos lugares donde el desarrollador puede adaptar el framework para construir aplicaciones diferentes dentro de una misma familia.

⊱ En un principio, el concepto hotspot es un concepto abstracto que describe un acuerdo de diseño entre el diseñador del framework y los desarrolladores; el primero define qué puede variar (hotspot) y qué no (frozenspot), el segundo acepta ese trato y utiliza puntos de extensión para personalizar la aplicación.

Puede variar en granularidad (depende de la variación permitida, un hotspot puede ser muy chico o muy grande) y luego, al llevarlo al código, pueden terminar siendo:

a. sólo un método;
b. clases a subclasificar, implementando un conjunto de métodos en cada una;
c. objetos que se configuran o componen.

Basicamente, hay que comprender que un hotspot no es, en principio, una cosa concreta del código, es un **concepto de diseño** (que luego se implementa en código de distintas maneras).

⁘ El concepto hotspot, de manera abstracta, hace referencia a un aspecto que se puede cambiar, reconocerlos nos permite diseñar mejor el framework y las aplicaciones que lo usan.

Instanciando hotspots

Cuando el desarrollador aprovecha/utiliza un punto de extensión del framework para definir un comportamiento específico se dice que “instancia un hotspot”

Un framework frecuentemente ofrece muchos hotspot. Tendrá hotspots que se instancian con herencia (whitebox) y otros con composición (blackbox).

En muchos casos, instanciar un hotspot puede requerir una combinación de acciones, de herencia y de composición (combinando ambos enfoques).

Puntos de extensión

¿Que son los puntos de extensión?

☒ El framework ofrece puntos de extensión que **permiten separar la implementación de la parte constante** (**frozenspot)** y las partes que varían (**hotspots).**

Los puntos de extensión tienen la forma de plantillas con ganchos (hooks) para que los programadores de aplicaciones puedan definir el comportamiento variable y adaptar el framework para construir aplicaciones diferentes dentro de una misma familia.

☒ Los puntos de extensión pueden implementarse en base a:

- herencia (por ejemplo, Template Method)
- composición (por ejemplo, Strategy)

Observación: Algunos patrones de diseño (particularmente el template method) son frecuentemente utilizados para introducir esos puntos de extensión.

hotspot ≠ puntos de extensión: un hotspot es un concepto de diseño (acá quiero permitir cambios, representa la variabilidad) y un punto de extensión es la forma concreta en código que permite implementar ese hotspot. (este método/interfaz permite hacerlo)

Caja blanca vs Caja negra

Estos conceptos se relacionan a "*¿cuanto hay que saber sobre la implementación interna del framework para utilizarlo?",* se trata de cómo personalizas (instancias el framework).

‣ Los frameworks suelen evolucionar: generalmente comienzan dependiendo mucho de la **herencia (caja blanca)** porque es más fácil de desarrollar para el diseñador del framework (auque es más desafiante usarlos para los desarrolladores), y con el tiempo se transforman hacia la **composición (caja negra)**, que es más desafiante de programar pero mucho más sencilla de usar para los desarrolladores.

→ Usar un framework de caja blanca es más facil de desarrollar para el diseñador del framework y más flexible, pero le lleva más trabajo al desarrollador final para utilizarlo (requiere entender diseño interno)

→ Usar un framework de caja negra es más facil de utilizar para el desarrollador final pero menos flexible y lleva más trabajo para el desarollador del framework

‣ La mayoria de los frameworks están en algún lugar en el medio, combinan ambos enfoques: Algunos usuarios los ven como caja negra y otros como caja blanca

‣ Un framework puede pasar de ser caja blanca a caja negra, o viceversa.

¿Que son los frameworks de Caja Blanca (whitebox)?

Son aquellos frameworks que **utilizan herencia en sus puntos de extensión**.

- El desarrollador personaliza el comportamiento usando herencia, creando subclases o sobreescribiendo métodos.
  - Al instanciar el framework, se hereda y completa agregando código
- Requieren conocer la estructura interna para saber dónde extender.
- En una primera etapa, cuando todavía el framework está evolucionando, hay que poder acceder, entender, estudiar la implementación del mismo, por lo tanto, se utiliza herencia en general y se dice que el framework es de caja blanca.
- El loop de control (indica quién decide el flujo de ejecución del programa, en frameworks, generalmente existe inversión de control y lo define el framework mismo) suele estar definido con Template Method.
  - El framework define el algoritmo general y el desarrollador personaliza partes del comportamiento mediante herencia y sobrescritura de métodos.

¿Cuales son los frameworks de Caja Negra (blackbox)?

Son aquellos frameworks que **utilizan composición en sus puntos de extensión**.

- El desarrollador personaliza el comportamiento usando composición, instanciando objetos, configurandolos, en lugar de crear nuevas subclases
  - No se toca el código del framework, se arma la nueva aplicación conectando objetos
- Para usar estos frameworks no se requiere conocer su implementación interna.
- Surgen en etapas más maduras del framework.
  - A medida que el framework es utilizado por muchas aplicaciones, los desarrolladores identifican variaciones comunes y las transforman en componentes configurables. De esta forma, en lugar de crear nuevas subclases mediante herencia, las nuevas aplicaciones pueden construirse mediante composición y configuración de objetos ya provistos por el framework.
- El loop de control es manejado internamente por el framework a través de un contenedor o configurador (no Template Method)

≠ caja blanca permite modificar comportamiento y caja negra permite elegir comportamiento

❔ **¿Por qué casi TODOS los frameworks grandes empiezan como caja blanca y evolucionan a caja negra?**

Al principio no se sabe con claridad qué aspectos van a variar. Cuando nace un framework todavía no se conocen todas las necesidades reales y no se puede identificar con precisión cuáles serán los hotspots importantes. Por eso, permitir extensión mediante herencia resulta más simple y flexible en las primeras etapas, ya que facilita experimentar con distintas variaciones.

A medida que muchas aplicaciones utilizan el framework, comienzan a repetirse patrones de variación y el diseñador logra tener hotspots bien identificados. Estas variaciones recurrentes se encapsulan y se transforman en puntos de extensión configurables.

De esta forma, el desarrollador ya no necesita heredar ni modificar el código interno del framework para especializar el comportamiento, sino que puede construir nuevas aplicaciones configurando y componiendo componentes ya provistos por el framework.

Ejemplo dado en clase:

Al comienzo, cuando se desarrolla una aplicación para una flota de taxis (por ejemplo, La Plata), todavía no se conocen todas las posibles variaciones futuras. Si luego se quiere adaptar para otra ciudad (Berisso), una estrategia común es separar el código común del código variable utilizando clases abstractas o patrones, creando subclases específicas para cada caso. Este enfoque requiere conocer el diseño interno del framework y extenderlo mediante herencia, lo cual caracteriza a los frameworks de caja blanca.

A medida que aparecen más clientes y se identifican variaciones recurrentes, los desarrolladores del framework refactorizan esas diferencias y las transforman en hotspots más explícitos y configurables. Así, en lugar de crear nuevas subclases, las nuevas aplicaciones pueden construirse configurando o componiendo objetos ya existentes, lo que hace que el framework evolucione hacia un modelo de caja negra, más fácil de usar.

Preguntas de autoevaluación (pueden entrar en final)

❔ **Cuando hablamos de reuso en el marco del desarrollo de software, ¿a qué nos referimos?**

El reuso implica no realizar todos los requerimientos desde cero, sino buscar soluciones que ya sabemos que funcionan. Se reusan conceptos e ideas, diseños o estrategias (como patrones de diseño), funciones, estructuras de datos, componentes/servicios (como mapas de Google) e incluso aplicaciones completas que se adaptan mediante plugins. Su objetivo es aumentar la productividad y calidad, reduciendo costos y tiempos de entrega

❔ **¿Cuáles son las dificultades que encontramos cuando intentamos reusar código?**

Falta de control: Se suelen reusar componentes ajenos; si estos cambian o se rediseñan, pueden obligar a reescribir nuestro código
Complejidad técnica: Crear software reusable es más costoso y difícil que el software tradicional
Curva de aprendizaje: Encontrar y aprender a usar componentes adaptables requiere un esfuerzo adicional

❔ **Cuando aplicamos patrones de diseño, ¿qué estamos reusando? ¿De qué manera los patrones de diseño atacan las dificultades del reúso?**

Al aplicar patrones, estamos reusando **estrategias de diseño** que ya fueron pensadas, probadas, mejoradas, maduradas y documentadas por especialistas para problemas particulares y, sobre todo, conocimiento y tiempo ya que se evita redescubrir soluciones.

Los patrones de diseño atacan las dificultades del reúso al encapsular soluciones de diseño que han demostrado ser efectivas, reduciendo la dependencia de código concreto y, por lo tanto, la pérdida de control frente a cambios. Asimismo, permiten manejar la complejidad del software reusable separando responsabilidades y definiendo puntos de extensión claros. Además, al ofrecer estructuras conocidas y un lenguaje común, disminuyen la complejidad técnica del diseño reusable y reducen la curva de aprendizaje necesaria para comprender y aplicar soluciones reutilizables.

❔ **Cuando utilizamos librerías de clases, ¿qué estamos reusando? ¿a qué casos apuntan? ¿De qué manera atacan las librerías de clases las dificultades de reúso?**

Reusamos clases independientes que resuelven problemas concretos sin importar el contexto. Apuntan a problemas puntuales y bien delimitados, son herramientas.

Atacan el reuso permitiendo que nuestro código mantenga el control y use solo lo que necesite. Encapsulan detalles díficiles y se aprendé solo lo que se necesita.

❔ **Cuando utilizamos frameworks orientados a objetos, ¿qué estamos reusando? ¿a qué casos apuntan? ¿De qué manera atacan los frameworks las dificultades de reúso?**

Reusamos diseño y **arquitectura** para una familia de aplicaciones, es decir, una aplicación semi-completa. Apuntan a familias de aplicaciones relacionadas (resuelven una problemática de dominio completa).

Atacan el reuso definiendo hotspot y frozenspot limitando el impacto de cambios, usan patrones de diseño ya probados y encapsulan la parte más compleja del dominio (hacen viable el reúso a gran escala). En cuanto a la curva de aprendizaje, es más alta que en librerías, pero una vez aprendido, se reutiliza en muchas aplicaciones (inversión inicial).

❔ **Por lo general, ¿es posible y lógico reusar, de forma aislada, solo algunas clases de un framework? ¿es posible y lógico utilizar solo algunas clases de una librería de clases? Justifique.**

A diferencia de las librerías, es **ilógico reusar clases aisladas** de un framework porque son altamente cohesivas y trabajan juntas para resolver un problema mayor; juntas conforman un todo.
Por otro lado, es lógico reusar solo algunas clases de una librería ya que las clases son independientes entre sí, por ende, menos acopladas entre sí y están pensadas para un uso puntual.

❔ **De acuerdo a el tipo de problema que atacan, ¿como se clasifican los frameworks?**

De infraestructura: Resuelven problemas técnicos comunes a cualquier programa (eficiencia, portabilidad) que el usuario final no suele percibir
De integración: Actúan como interfaz entre distintas partes, se utilizan para integrar componentes de aplicación distribuidos.

De aplicación: Capturan diseño e implementación para dominios específicos (ej. una flota de taxis), basándose en las necesidades del usuario final

❔ **¿A que nos referimos con los términos hot spot y frozen spot?**

Frozen spot (Punto congelado): Son las partes del framework que se mantienen constantes en todas las aplicaciones construidas con él; definen el esqueleto y el orden de los pasos
Hot spot (Punto caliente): Son los puntos de variabilidad donde el usuario decide cómo implementar algo de forma diferente; representan la flexibilidad del sistema

❔ **¿A qué nos referimos con "instanciar un framework"? ¿Qué sería "instanciar un hostpot"?**

Instanciar un framework: Es el proceso de iniciar la estructura básica (el frozenspot), como instanciar un `GameBoard` en el ejemplo de los robots
Instanciar un hotspot: Es completar los "huecos" de variabilidad, ya sea mediante **herencia** (redefiniendo métodos ganchos) o **composición** (configurando objetos específicos)

❔ **¿En qué se diferencia la visión que tienen de un framework: a) los desarrolladores del framework, b) quienes usan/instancian el framework?**

Desarrollador: Se enfoca en decidir qué será constante y qué variará, dejando reglas claras y ganchos para el futuro
Usuario: Siente que está "completando el diseño de otro", rellenando los huecospara que el sistema semicompleto se transforme en uno completo

❔ **¿En qué se diferencia un framework blackbox de uno whitebox? Un framework, ¿cae exclusivamente en una de esas categorías?**

Whitebox (Caja Blanca): Basados en **herencia**. Requieren que el usuario conozca el diseño interno y el código para saber qué métodos sobrescribir
Blackbox (Caja Negra): Basados en **composición**. El usuario solo instancia y configura objetos (callbacks) sin necesidad de mirar el código interno.

Evolución: No son categorías exclusivas; los frameworks suelen empezar como whitebox y, al madurar, evolucionan hacia blackbox

❔**Desde la perspectiva de quien "hace" el framework, ¿qué implica optar por whitebox, o blackbox? Piense por ejemplo en las facilidades y las dificultades que encuentra en cada caso.**

Whitebox

√ Más facil y rápido de desarrollar inicialmente (menor esfuerzo).

√ No hace falta anticipar todos los casos de uso: El desarrollador no tiene que pensar en como los usuarios van a querer extender o cambiar el framework ya que el usuario puede meterse adentro y adaptar el comportamiento aunque el desarrollador no lo haya pensado.

× Como el código del usuario depende de la estructura interna, cualquier cambio en clases base que haga el desarrollador pueden romper subclases existentes y generar efectos colaterales

× Evolucionar y refactorizar el framework se vuelve costoso (debido al item anterior)

× El desarrollador se queda atado al diseño original

Blackbox

√ El desarrollador mantiene control sobre la arquitectura, puede refactorizar la implementación interna sin afectar a usuarios

√ Mejora el encapsulamiento

√ La evolución del framework es más segura

× Debe identificar y definir correctamente todos los hotspots, interfaces, configuraciones posibles

× Requiere más esfuerzo en el diseño inicial

× Es difícil anticipar todas las variantes necesarias
observación: Con usuario me refiero a desarrollador final que usará el framework.

❔ **Desde la perspectiva de quien "usa" el framework, ¿qué implica que el mismo sea whitebox, o blackbox? Piense por ejemplo en las facilidades y las dificultades que encuentra en cada caso.**

Whitebox

√ Alta flexibilidad: El usuario puede sobreescribir métodos, redefinir comportamientos internos, acceder a protected methods, hooks, etc.

√ Posibilidad de modificar comportamientos complejos

√ No está limitado a puntos de extensión explícitos

× La curva de aprendizaje es alta, el usuario debe estudiar la implementación y aprender qué se está heredando, entender jerarquías de clases, métodos heredados, comportamiento interno

× Tiene dependencia fuerte del diseño interno del framework

× Tiene riesgo de errores por mal uso de la herencia

Blackbox

√ Es más facil de usarlo

√ Menor curva de aprendizaje, se interactua mediante interfaces, configuración.

√ El código del usuario tiene menor impacto frente a cambios internos

✕ Menor flexibilidad

✕ El usuario extiende solo lo que el framework permite

❔ **La cantidad de "puntos de variabilidad" de un framework (es decir, sus hotspots) depende de cuan complejo es el dominio que ataca (es decir, de cuantas cosas podrían cambiar de aplicacion a aplicación). Por lo general, un framework tiene más hotspots que los que el usuario promedio requiere (piense que quien hizo el framework tuvo en cuenta las expectativas de muchos de ellos). ¿Siempre es necesario instanciar (configurar, subclasificar) todos los hotspots o hay algo que el diseñador del framework puede hacer para reducir el trabajo de instanciación del mismo en los casos más simples?**

No, no siempre es necesario configurar todos los hotspots, si así fuera, incluso en los casos simples, el frameworks podría considerarse pesado y dificil para aprender/usar.

Como un framework suele tener más hotspots de los necesarios, para reducir el trabajo del usuario, el diseñador del framework puede ofrecer **implementaciones por defecto** en los ganchos (*hook methods*), de modo que el usuario solo tenga que intervenir en los puntos que realmente desea cambiar.

Por ejemplo, pueden utilizarse configuraciones predefinidas o convenciones que permiten cubrir los casos más simples, dejando que la especialización aparezca solo cuando se necesita.

O también, en caso de encontrarse los hotspot en métodos, dejar que por defecto no hagan nada y que despúes, en caso de ser necesario, se redefinan.

❔ **¿Que patrones de diseño, de los que hemos visto en clase, se encuentran comúnmente en los hotspots de un framework? ó ¿Qué patrones de diseño, de los vistos en clase, son generadores de hotspots?**

-Los hotspots de un framework suelen implementarse mediante patrones de diseño que separan los aspectos variables de los constantes.

-Un patrón genera hotspots cuando permite variar comportamiento sin modificar el código del framework.

El patrón **Template Method** es el generador de hotspots por excelencia mediante herencia. Define el esqueleto de un algoritmo y delega los pasos variables a métodos abstractos o hooks.

El patrón Strategy genera hotspot mediante la composición.

❔ **Los conceptos de template y hook (plantilla y ganchos) los encontramos en el template method, en el marco de una relación entre una clase y sus subclases. También se lo puede encontrar en una relación de composición. Ambas situaciones son comunes en los hotspots de un framework. ¿De que manera afecta una y la otra alternativa al comportamiento runtime de las aplicaciones que instancia el framework/hostpot?**

Herencia: El comportamiento queda fijo al instanciar el objeto; **no se puede cambiar en tiempo de ejecución** (runtime) sin crear una nueva instancia
Composición: Permite **cambiar el comportamiento en tiempo de ejecución** simplemente reemplazando el objeto componente (el gancho) por otro

❔ **Le proponen hacer hacer una aplicación en un dominio que es nuevo para usted. Le aclaran que a esa le seguirán muchas parecidas. Usted piensa automáticamente en reúso y en cuanto tiempo y dinero ganará a futuro si puede reusar. Enfoca toda su fuerza en, como primer paso, construir un framework. ¿Tiene sentido? Si cree que si, ¿cómo lo haría? Si cree que no, cuáles cree que son los pasos para eventualmente poder reusar.**

Un framework es el **resultado de refactorizar** aplicaciones existentes para abstraer lo común. Por lo tanto, no tiene sentido construir uno para un dominio nuevo sin antes haber desarrollado aplicaciones similares que permitan identificar realmente qué debe ser un frozenspot y qué un hotspot.

El paso lógico es: construir aplicaciones, notar patrones, refactorizar hacia un framework.

❔ **¿Cómo cree usted que se relacionan los temas: patrones de diseño, refactoring (y refactoring to patterns), frameworks, y unit testing?**

Estos temas están relacionados dentro del proceso de evolución del software. La relación entre ellos puede verse como un ciclo

Unit testing → permite refactoring seguro → refactoring introduce patrones → patrones permite crear frameworks reutilizables

Gracias a los test unitarios podemos realizar refactoring de manera segura ya que los test nos ayudan a garantizar que el comportamiento del sistema no cambia al mejorar la estructura interna del código.

Durante el refactoring, suelen identificarse estructuras que pueden expresarse mediante patrones de diseño (refactoring to patterns). A medida que el código crece, aparecen problemas (como duplicación de código, acoplamiento, etc.) que luego de refactorizarse permiten identificar soluciones recurrentes, esas soluciones suelen ser patrones, lo cual se conoce como Refactoring to patterns.

Estos patrones permiten separar los aspectos constantes de los aspectos variables mediante puntos de extensión y, por lo tanto, constituyen la base para construir frameworks. Los frameworks encapsulan diseño probado, por eso suelen estar llenos de patrones.

A su vez, los frameworks bien diseñados (al igual que los patrones de diseño) promueven arquitecturas desacopladas y con separación de responsabilidades, lo que mejora la testabilidad del sistema y facilita la realización de testing unitario.

Logging

¿Qué es logging?

Logging es dejar registros (logs) de lo que hace un programa mientras se ejecuta.

Son basicamente mensajes que permiten entender qué está pasando dentro del programa, como:

- Reportar eventos importantes, errores y excepciones
- Indicar pasos críticos en la ejecución
- Indicar el incio y fin de operaciones complejas o largas

Los logs no reemplazan al testing.

¿Por qué no alcanza con system.out.println()?

Si bien se puede hacer algo del estilo:

```
System.out.println("Entré al método X");
```

No se puede filtrar facilmente qué mensajes ver en caso de haber muchos, si es importante o es uno trivial, si no quieren verse más los mensajes hay que borrarlos o comentar el código...

Entonces aparecen frameworks de logging.

¿Qué es un framework de logging?

Un framework de logging es una herramienta que permiten:

- Estandarizar la práctica: Logra que todos loguean de la misma manera, siguiendo reglas comunes, evitando que los programadores realicen mensajes distintos para lo mismo, entre otras cosas.
- Activar o desactivar logs selectivamente, incluso sin modificar el código
- Enviar los reportes a distintos formatos y destinos (txt, JSON, XML, archivos, pantalla, ...): Los mensajes de log pueden enviarse a distintos lugares y presentarse en distintos formatos sin cambiar el código
    - Por ejemplo, durante el tiempo de desarollo se muestra todo en pantalla y en producción se guardan errores en un archivo, sin cambiar el código.
- Oculta los detalles de implementación: El framework funciona mayormente como una caja negra para el desarrollador.
    - Desde el punto de vista de la aplicación, solo se necesita pedir un Logger y enviarle un mensaje .log(), toda la complejidad de cómo ese mensaje se formatea, se filtra por niveles, queda encapsulada dentro del framework.

Java Logging Framework

Java Logging Framework está incluido en el SDK (software development kit) de Java.

¿Como funciona?

» El log se hace enviando mensajes a objetos Logger. No se imprime nada directamente, se manda el mensaje a un objeto Logger que es el punto de entrada al sistema de logging.

» Se encarga del registro de Loggers. Existe un LogManager global que sabe qué Loggers existen, mantiene la jerarquía y devuelve el mismo Logger cuando lo pedís.

» Los loggers se organizan en un espacio jerárquico de nombres

  » Heredan propiedades y propagan mensajes (un mensaje logueado en un logger hijo puede subir al padre y terminar siendo manejado por sus handlers)

» Los mensajes de error se asocian a niveles (cada nivel indica que tán importante es)

  » Cada mensaje tiene un Level (INFO, WARNING, etc.)

  » Permite filtrar mensajes por nivel

» Se puede extender (nuevos formatos, destinos, filtros, etc).

  » Si lo que trae Java no alcanza, se puede crear tu propio Formatter, Handler, Filter, etc.

» En cualquier método, se puede recuperar y utilizar un logger.

¿Qué define el framework?

» Define como se crean, organizan y recuperan los objetos Logger

```
//crear
Logger.getLogger("mi.app");
//los loggers tienen nombres, esos nombres forman una jerarquia, un árbol
app
app.users
app.users.auth
//como se recuperan
Logger.getLogger("app.users"); //siempre se recupera el mismo
```

» Define como se configuran los objetos Logger

  » La configuración suele ocurrir al inicial la aplicación (y se mantiene globalmente)

» Define como se activan y desactivan los objetos Logger

» Define a qué prestan atención y a qué no

  » Cada logger tiene un nivel mínimo e ignora los mensajes menos importantes. Por ejemplo, “este logger solo presta atención a WARNING, si llega un INFO se ignora”

» Define como se formatean los logs

  » Permite ajustar el formateo de los mensajes (como se ven los mensajes, XML, texto, etc.) → Esto lo decide un Formatter

» Define a dónde se envían los logs

  » Permite enviar mensajes a consola, archivos, red (sockets) → Esto lo decide un Handler, el framework trae Handlers ya hechos (ConsoleHandler, FileHandler, SocketHandler)

Ejemplo básico

```java
public class Sandbox {
    public static void main(String[] args) throws IOException {
        Logger.getLogger("app.main").addHandler(new FileHandler("log.txt"));
        Logger.getLogger("app.main").log(Level.INFO, "App iniciada");
        try {
            // Acá que hace algo que "podría" resultar en una excepción
            int explodesForSure = 1 / 0;
        } catch (Exception ex) {
            Logger.getLogger("app.main").log(Level.SEVERE, "Explotó!", ex);
        }
        Logger.getLogger("app.main").log(Level.INFO, "App terminada");
    }
}
```

→ Se recupera o crea (si no existe) un objeto Logger asociado al nombre "app.main", se agrega un FileHandler para que, además de sus destinos por defecto, escriba los mensajes en un archivo "log.txt"

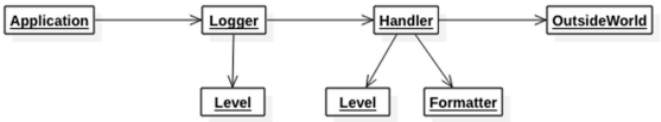
→ Se emite el primer mensaje, utilizando Level.INFO para indicar que es un mensaje informativo sobre el progreso de la app

→ El código realiza una división por cero, lo cual genera una excepción aritmetica, dentro del catch se recupera el mismo objeto logger (no hace falta utilizar una variable) que registrará la excepción (se pasa un tercer parámetro para guardar detalles técnicos del error ocurrido)

→ Finalmente, se registra que la aplicación terminó

En resumen, este código demuestra cómo reemplazar un simple `System.out.println` por un sistema estructurado que clasifica la importancia de los mensajes (INFO vs SEVERE) y redirige la salida a un archivo persistente, sin perder los detalles de las excepciones y logrando verificar el ciclo completo de la ejecución desde el inicio hasta el fin.

Partes básicas (visibles)



Logger →

- Es el objeto al que le pedímos que emita un mensaje de log.
- Se pueden definir tantos como se necesiten

```
//Son instancias de la clase Logger
//Se obtienen/crean de la siguiente manera:
Logger.getLogger(String nombre)
//Para emitir algún mensaje
log(Level, String)
```

- Cada uno con su filtro y handler/s (un Logger puede tener asociado muchos handlers)
- Se organizan en un árbol basado en nombres.
  - Por ejemplo, ecommerce es padre de ecommerce.customers.
  - Loggers ueden heredar configuración de su padre (handlers y filters)

Handler →

- Recibe los mensajes del Logger y es el encargado de enviar el mensaje a donde corresponda.
- Dependiendo de adónde quiera enviar la información, existen distintos tipos como: Instancias de MemoryHandler, ConsoleHandler, FileHandler, o SocketHandler.
- Puede filtrar por nivel.
- Tiene un Formatter.

Level →

‣ Indica la importancia de un mensaje y es lo que mira un

Logger y un Handler para ver si le interesa.

‣ Cada vez que se intenta loggear algo, se debe indicar el nivel, los loggers y handler comparan el nivel de cada mensaje con el suyo para decidir si les interesa o no.

  ‣ Si te interesa un nivel, también te interesan los que son más importantes que ese

‣ Hay niveles predefinidos, en variables de la clase Level: Level.SEVERE, Level.WARNING, Level.INFO, etc.

Formatter →

‣ Determina como se ”presentará” el mensaje

‣ Recibe un mensaje de log (un objeto) y lo transforma a texto.

‣ Cada Handler tiene asignado un Formatter. Por ejemplo, los `FileHandler` suelen usar `XMLFormatter` por defecto, mientras que los `ConsoleHandler` usan `SimpleFormatter` (texto plano)

Observación: **Flujo de trabajo:** El proceso sigue esta secuencia lineal:

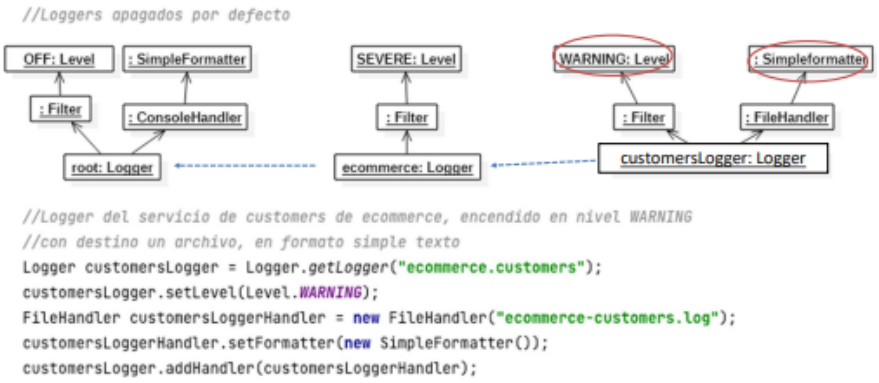
Aplicación → Logger → Handler (usa Formatter) → Mundo Exterior

---

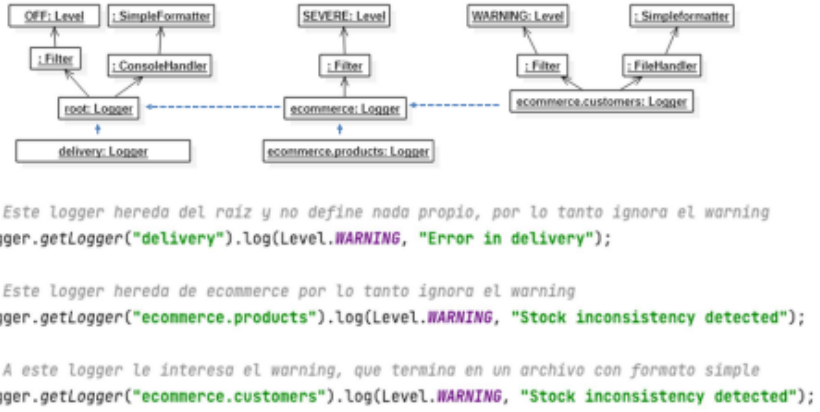
Ejemplo avanzado como caja negra

- Se obtiene el logger raíz `""` y se apaga completamente con `Level.OFF`. Esto significa que, por defecto, ningún logger en el sistema emitirá mensajes a menos que se diga lo contrario.
- Se crean loggers, organizados por nombre, formando un árbol jerárquico. Cada uno es un objeto Logger distinto, pero relacionados.
- Se configura el logger `ecommerce` con nivel `SEVERE`. Esto implica que todo el módulo de comercio electrónico solo reportará errores graves. Según el diagrama, este logger tiene asociado un `ConsoleHandler` (salida a pantalla)

```
root
 └── ecommerce
     └── ecommerce.customers
```



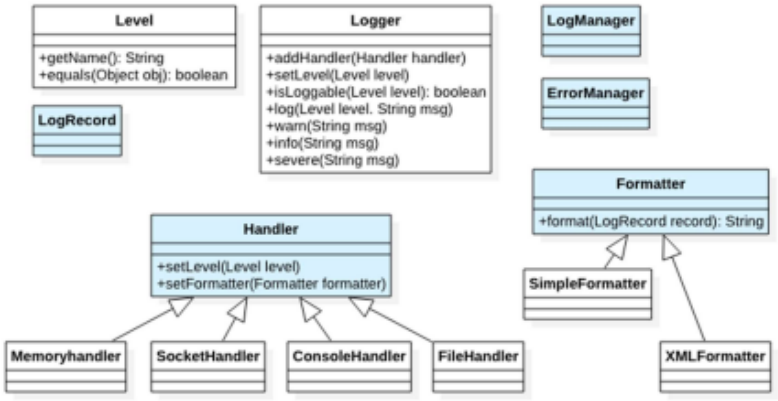
- **El Hijo ("ecommerce.customers") es** específico para el sub-módulo de clientes. Se baja la exigencia a `WARNING`. A este módulo sí se le permite reportar advertencias, no solo errores graves. Se le agrega un `FileHandler` para guardar los datos en un archivo llamado `ecommerce-customers.log` con un formato de texto simple (`SimpleFormatter`)



El único log registrado será el "**ecommerce.customers**" ya que fue configurado explícitamente (en el paso 1) para aceptar nivel `WARNING`. El mensaje pasa el filtro y se escribe en el archivo configurado.

> Observación → Al tratarlo como "caja negra", nos enfocamos en cómo configurar y usar los objetos (`Logger`, `Handler`, `Level`) sin preocuparnos por su código interno.

Lo que no se ve (desde afuera de la caja)



> Se pueden ver los detalles de implementación y los componentes internos que el framework gestiona automáticamente para que el desarrollador no tenga que hacerlo

- LogRecord: Objeto que representa un mensaje.
  - Cuando se escribe `logger.log(Level.INFO, "Hola")`, no se está pasando texto simple por todo el sistema. El framework encapsula esa información en un objeto llamado `LogRecord`. Este objeto viaja a través de los filtros y formateadores.
- LogManager: Es el objeto que mantiene el árbol de loggers en memoria; hay un solo LogManager (global).
  - Cuando se pide un logger con `Logger.getLogger("nombre")`, quien realmente hace el trabajo pesado es el `LogManager`.
- Filter: Logger y Handler tienen un Filter para determinar si algo les intersa o debe ser descartado (el Filter conoce al Level).
- ErrorManager: los Logger pueden tener un ErrorManager para lidiar con errores durante el logging, evitando que la aplicación caiga solo porque fallo el log.
- Clases abstractas Handler y Formatter
- ... y todas las relaciones entre ellos.

"Lo que no se ve" son las piezas que permiten la **abstracción**. Gracias a ellas, solo se ve una interfaz sencilla (`log()`), mientras el framework coordina internamente la creación de registros, la gestión del árbol jerárquico y el manejo de errores.

¿Qué seria lo que no cambia (frozenspot)?

→ Diseño/propuesta sobre como integrar logging

→ Estrategia general: La forma de integrar el logging siempre es buscando un logger para configurarlo o pedirle que emita un mensaje

→ Los componentes: Siempre existiran las abstracciones de Logger, Handler, Formatter, Filter, Level

→ Cómo se organizan los loggers: Los loggers siempre se organizan en un árbol gestionado globalmente (por el `LogManager`)

→ Las configuraciones se heredan

→ Qué pasa cuando le digo log() a un logger (la secuencia de colaboraciones)

¿Qué seria lo que si cambia (hotspot)?

Se puede ver de dos formas:

Como una configuración (Uso como Caja Negra):

→ Cuántos y cuales loggers utilizo

→ Qué logger "hereda" de cuál

→ Qué niveles de importancia le interesan a cada logger (su filter)

→ Qué handlers se asocian a cada logger (archivo, consola, red)

→ Qué formatter tiene cada handler y que le interesa (su filter)

Como una extensión (Uso como Caja Blanca):

Si la configuración no es suficiente, se puede cambiar el comportamiento escribiendo nuevo código ("lo que cambia") que se integra en la estructura ("lo que no cambia")

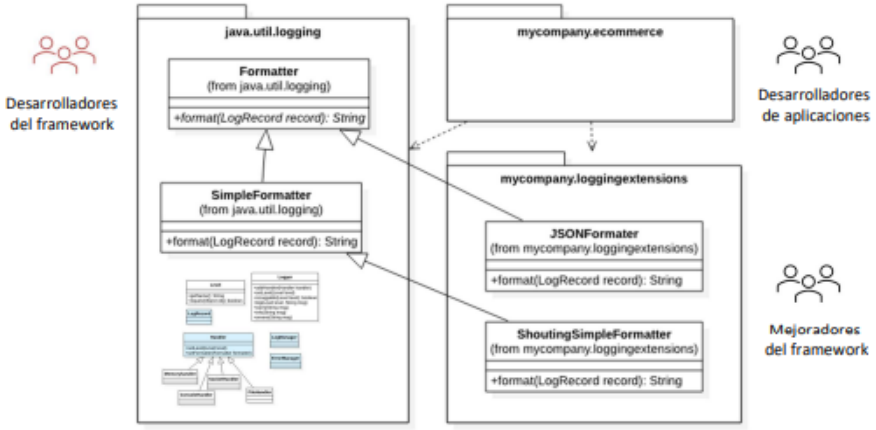
→ Se puede extender el framework heredando en los puntos de extensión previstos

Entonces, mirando adentro, se pueden agregar nuevas clases de Formater, Handler y Filter.

∴ **Nuevo Formatter**: Se puede crear una clase propia (ej. `JSONFormatter` o `ShoutingSimpleFormatter`) subclasificando la clase abstracta Formatter o alguna de sus subclases.

```java
public class ShoutingSimpleFormatter extends SimpleFormatter {
    @Override
    public String format(LogRecord record) {
        // SHOUTING WITH ALL UPPERCASE
        return super.format(record).toUpperCase();
    }
}
```

```java
public class JSONFormatter extends Formatter {
    @Override
    public String format(LogRecord record) {
        // Do whatever necesssary to represent record as
        // a JSON formatted string and return it
        return "...";
    }
}
```

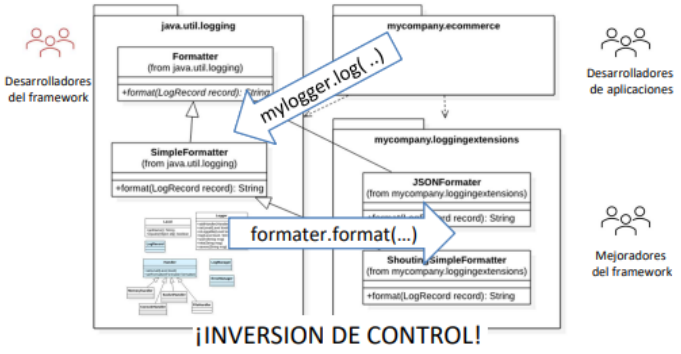


- **Nuevo Handler:** Se puede crear un nuevo handler subclasificando la clase abstracta Handler o alguna de sus subclases.
- **Nuevo Filter:** Implemento la interfaz Filter.

Observación → Aparece el concepto de ***Inversión de Control***

El código que no cambia es el que tiene el control del flujo y llama al código que sí cambia (las clases personalizadas como un formateador JSON) cuando es necesario.

> ¿Quien envía el mensaje format() a las instancias de nuestras clases JSONFormater y ShoutingSimpleFormater?



Al agregar la subclase como ShoutingSimpleFormater se sobreescribe el método format(), pero nunca se llama manualmente a ese método format() en la aplicación main, el framework mantiene el control del flujo de ejecución, cuando llega un log se debe pasarlo a un handler y luego formatearlo.

Entonces, cuando la aplicación corre, el framework detecta que se debe formatear un mensaje. El framework envia el mensaje format(...) a la instancia de la subclase.

Rta: El código interno del framework (los desarrolladores del framework) llamando al código nuevo que se escribió.

Inversión de control y hotspots

¿Qué es la inversión de control?

→ La inversión de control es la **caracteristica principal de la arquitectura en tiempo de ejecución (run-time) de un framework**.

→ Esta característica permite que los pasos canónicos de procesamiento de la aplicación (aquellos pasos que son fijos y comunes a todas las aplicaciones construidas con él) sean especializados por objetos tipo manejadores de eventos a los que invoca el framework como parte de su mecanismo reactivo de despacho.

> → Quiere decir que el framework define el comportamiento/estructura general pero le permite al desarrollador personalizarlo mediante objetos (definidos por el desarrollador) que reaccionan a eventos; cuando ocurre un evento, estos objetos son llamados por el framework como parte de su mecanismo interno de distribución de trabajo.
>
> → El framework deja hotspots que son **puntos de extensión** donde se espera que el desarrollador agregue código propio, para modificar el comportamiento (sin cambiar el códgio del framework), que luego será invocado por el framework (acá entra el concepto de inversión de control, el control del flujo principal sigue siendo del framework)
>
> De esta manera, el usuario puede personalizar el comportamiento sin controlar el flujo principal.

⁜ La idea central es: El framework controla el flujo principal, no el programa, y llama al código del desarrollador cuando corresponde. No se puede decidir como se llaman ciertas cosas, solo se pueden escribir piezas (se implementan hotspots) que el framework va a usar/invocar cuando se requieran.

Plantillas y ganchos

Las plantillas (templates) y los ganchos (hooks) se pueden utilizar en un framework para **separar lo que se mantiene constante** (frozenspot) **de lo que varía** (hotspots).

¿Que es una plantilla (template)?

Es una estrategia de programación utilizada para representar los aspectos comunes de un sistema.

» En un framework, las plantillas:

> » Suelen estar bajo el control del desarrollador del framework
>
> » Se utilizan para implementar los aspectos constantes (el **frozenspot**) de la arquitectura.
>
> » Su función principal es **definir los pasos que deben ejecutarse** y **garantizar que se cumpla el orden en que deben ejecutarse** en un proceso o algoritmo, es decir, garantizar el flujo principal de ejecución.

¿Qué es un gancho (hook)?

Representa los **puntos de extensión (o hotspots)** dentro de un framework.

» En un framework:

> » El desarrollador del framework delega en el desarrollador (usuario) la implementación o instanciación de estos ganchos para especializar el comportamiento de la aplicación.
>
> » Es el punto donde el framework "engancha" código del usuario durante la ejecución.

Entonces…

→ Las plantillas definen el flujo principal y los ganchos definen dónde se puede cambiar algo dentro del flujo (luego todo será ejecutado por el framework, pero te deja intervenir en el hook)

→ Los ganchos permiten que los pasos comunes definidos por la plantilla puedan personalizarse (sin modificar la estructura general)

```
template():
   paso fijo A
```

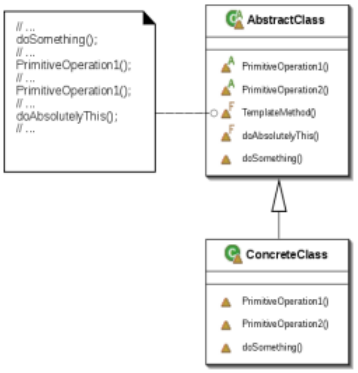
```
    hook_personalizable()
    paso fijo B
```

Existen dos formas principales de implementar los ganchos y las plantillas:

Para separar la plantilla (aquello que se mantiene constante) de los ganchos (aquello qeu varía) puede utilizarse **herencia o composición**

Herencia →

- La plantilla se implementa en una clase base abstracta
- Los ganchos se implementan en sus subclases



El patrón **template method** implementa plantillas y ganchos con herencia (y mensajes a self).
👉 La clase base define el algoritmo general (template method) y llama a métodos que pueden ser especializados.

¿Cuáles son realmente los ganchos dentro de un template method?

Los ganchos (hooks) en el metodo plantilla serán aquellos métodos que la plantilla llama y que **pueden ser redefinidos** por las subclases.

→ Métodos abstractos → No tienen implementación en la clase base, representan extensión obligatoria

→ Métodos hook → Tienen implementación por defecto (a veces vacía) en la clase base, representan extensión opcional

```
class TemplateBase: //para cambiar algo se crea una subclase
    template():
        paso1()
        hook()   ← redefino en subclase
        paso3()
```

Composición →

- Un objeto del framework implementa la plantilla
- Los ganchos se implementan en las partes, es decir, en otros objetos que colaboran con el objeto que implemento la plantilla (objetos colaboradores)

En lugar de crear subclases, el desarrollar crea objetos que implementan interfaces o comportamientos específicos y los configura o conecta con el framework. De esta manera, se puede especializar el comportamiento mediante la configuración y composición.

```
class Template:   //para cambiar algo, se pasa a otro objeto que sabe hacerlo
    def __init__(self, strategy):
        self.strategy = strategy

    template():
```

```
        paso1()
        strategy.hook()   ← objeto externo
        paso3()
```

Herencia vs Composición

El template method asegura que se cumplan los pasos y se programa "por diferencia" implementando los hooks mediante subclases, utilizando herencia	El template (la plantilla) asegura que se cumplan los pasos, pero delega la implementación de los hooks a objetos externos, utilizando composición
Es mas simple cuando hay pocas variantes y/o pocas combinaciones. Si hay muchas variantes y/o combinaciones, crece mucho la jerarquia de clases y puede haber duplicación de código	Evita la duplicación de código y el creciente numero de clases cuando existen muchas alternativas y combinaciones posibles (no se necesita heredar del framework para variar el comportamiento)
Los hooks tienen acceso a todo el comportamiento heredado de la clase abstracta (variables de instancia, métodos protegidos, etc.), es muy flexible.	Al implementar los hooks, deben recibir por parámetro lo que necesiten, no tienen acceso a las variables de instancia ni a métodos privados del objeto ni de los otros componentes. El acoplamiento es menor.
No se puede cambiar el comportamiento en tiempo de ejecución, suele quedar fijo.	Se puede cambiar el comportamiento en tiempo de ejecución sin mayor dificultad

---

Ejemplo conductor: Roborally, el juego de mesa

Es un framework que permite generar distintas variantes de juegos de luchas de robots.

Frozenspot

» Un tablero en forma de grilla

» Varios robots, los cuales se ubican en alguna celda del tablero

» El juego se organiza en pasos, cada paso del juego se corresponde un paso de cada uno de los robots en el tablero

» Cada paso de un robot sigue estas acciones en orden

  » moverse en el tablero

  » consumir energía (actualizar su registro de energía)

  » disparar su armamento

  » recolectar lo que pudiera haber en la celda del tablero que ocupa

Hostpots

Algunos hotspots permiten variar caracteristicas de robots.
» locomoción: este hotspot permite definir de que forma se moverá el robot. Dos alternativas para este hotspot son:

  » locomoción con orugas (caterpillar) que permite subir grandes pendientes

  » locomoción sobre colchón de aire (overcraft) que permite circular por zonas pantanosas.

» fuente de energía: este hotspot permite definir de que forma el robot obtendrá/almacenará/consumirá su energía. Dos alternativas para este hotspot son:

  » los paneles solares con baterías (muy estables pero sensibles a la falta de luz solar)

  » el reactor nuclear (poco estable, pero garantiza la provisión energética)

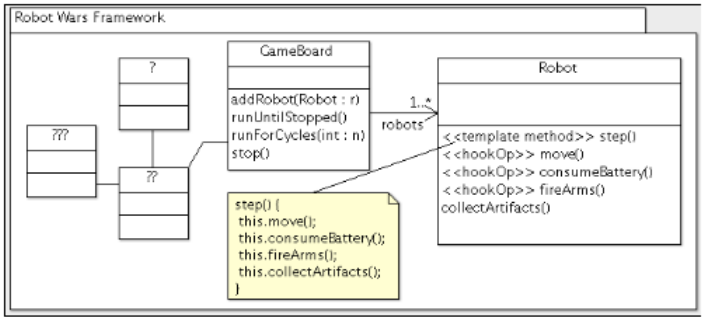
» armamento: este hotspot permite definir de que forma el robot atacará a otros. Dos alternativas para este hotspot son:

  » el láser (mucho alcance y precisión pero poco poder de daño)

  » las bombas (poco alcance y precisión, y mucho poder de daño)

Diseño

Las clases que nos interesan



Los pasos básicos para construir un juego con nuestro framework son:

1) se instancia el GameBoard (lo que resultará en la instanciación y configuración de muchos otros objetos que hacen al frozenspot del framework).

2) se instancian varios Robots y se agregan al juego enviando el mensaje *addRobot()* al *gameBoard*.

3) se inicia el juego enviando al *gameBoard* el mensaje *runUntilStopped()* . Si se quiere detener el juego se envía al gameBoard el mensaje stop().

- Alternativamente se puede activar el juego solo por un determinado numero de pasos enviando al gameBoard el mensaje runForCycles(int n).

El método plantilla

En cada paso del juego (en cada ciclo) el gameBoard recorre su colección de robots y a cada uno le envía el mensaje *step()*.
- El método *step()* es donde se garantiza que el robot efectúe las acciones establecidas en las reglas del juego (frozenspot), en el orden establecido.

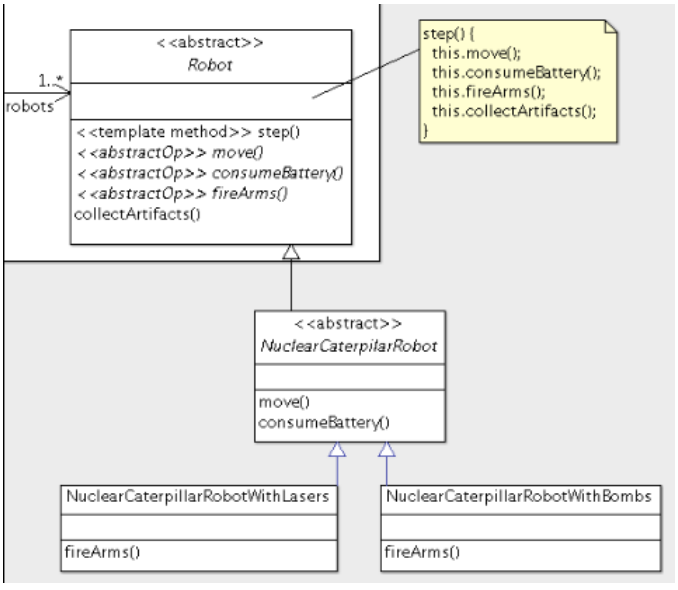
Implementado con herencia

Propone **abstraer los pasos canónicos de la operación en un método implementado en la clase abstracta y redefinir los pasos variables en sus subclases**.

De esta forma, la herencia es el mecanismo principal para separar lo que es constante, de lo que varía.

El método plantilla (template method) es parte del frozenspot mientras que las operaciones abstractas (o los ganchos) dan lugar a la implementación de los hotspots.

- En nuestro ejemplo el método implementado en la clase abstracte es el step()

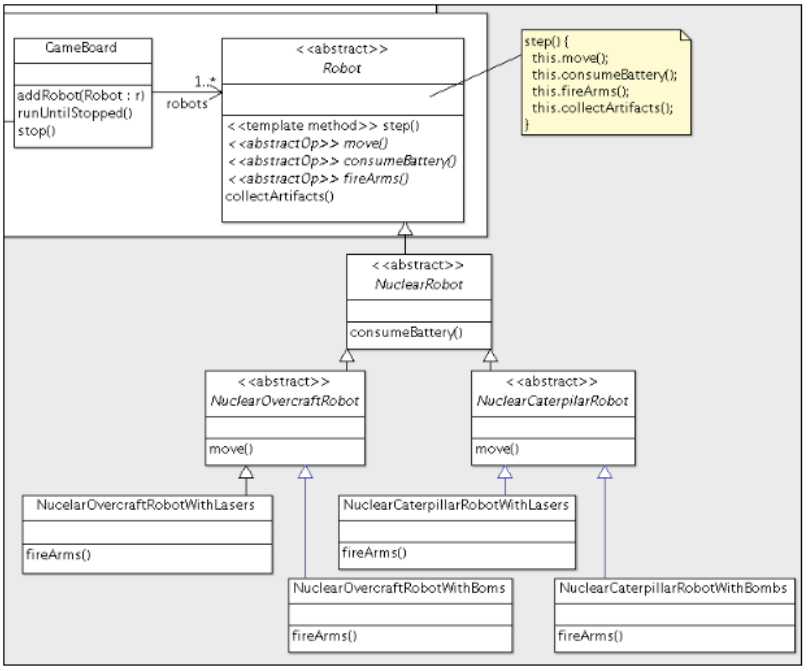


Ejemplo: Tenemos dos tipos de Robots que pueden agregarse a nuestro Juego. Los robots con orugas, alimentados con energía nuclear y armados lasers (NuclearCaterpillarRobotWithLasers) y los robots con orugas, alimentados con energía

nuclear y armados bombas (NuclearCaterpillarRobotWithBombs).

Ambas clases tendra el mismo comportamiento de locomoción (orugas) y fuente de energía (nuclear), se abstrae en la clase abstract NuclearCaterpillarRobot este comportamiento común. Por otro lado, en las subclases concretas quedan implementación del método fireArms(), dado que difieren una de otra

- *¿Qué sucede a medida que aparecen nuevas alternativas para cada hostpot y si se intentan combinar arbitrariamente?*



Observe que el comportamiento de armamento se encapsula en el método fireArms().

Necesitamos que una de las subclases de NuclearCaterpillarRobot implemente fireArms() para obtener comportamiento de armamento con lasers (NuclearCaterpillarRobotWithLasers). Un método fireArms() idéntico se requiere en una de las subclases de NuclearOvercraftRobot. Algo similar ocurre con el método fireArms() correspondiente a la implementación de armamento con bombas.

No hay forma de abstraer ese comportamiento en una clase abstracta (para evitar duplicar código) sin reformular toda la jerarquía y que ocurra algo similar con otro de los comportamientos variables.

Como no se puede lidiar bien con la combinación y abstracción de comportamiento ortogonales, resulta en una explosión en el numero de subclases y en la duplicación de código.

Esto evidencian las limitaciones de la herencia como estrategia para separar plantillas y ganchos. En situaciones como estas puede explorarse la composición como una estrategia alternativa.

- ¿Que debo hacer si aparece una nueva fuente de energía (por ejemplo, Paneles solares con baterias)? ¿Cuantas y cuales clases debo agregar en caso de querer todas las variantes de robots posibles para este nuevo tipo de fuente de energía?

Para agregar esta nueva fuente y cubrir **todas las variantes posibles**, te enfrentarías al problema de la **explosión de la jerarquía.**

Si asumimos que existen 3 formas de avanzar (Orugas, Ruedas, Overcraft) y 2 tipos de armas (Láser, Bombas), deberías agregar:
- **Una clase abstracta intermedia:** por ejemplo, `SolarRobot`, para definir el método `consumeBattery()` común a esta fuente
- **Seis clases concretas (hojas):** para cubrir todas las combinaciones (ej. `SolarOvercraftLaserRobot`, `SolarOvercraftBombRobot`, `SolarCaterpillarLaserRobot`, etc.)
En total, tendrías que agregar **7 clases nuevas** solo para este tipo de energía.

- ¿Puedo cambiarle, a un robot existente, el sistema de armas sin tener que instanciar el robot de nuevo?

Si bien el comportamiento constante se separa del comportamiento variable en métodos diferentes, al instanciar los robots, ambos comportamientos (grupos de métodos) pasan a formar parte de un único objeto (el robot con los métodos de su clase y los que hereda).

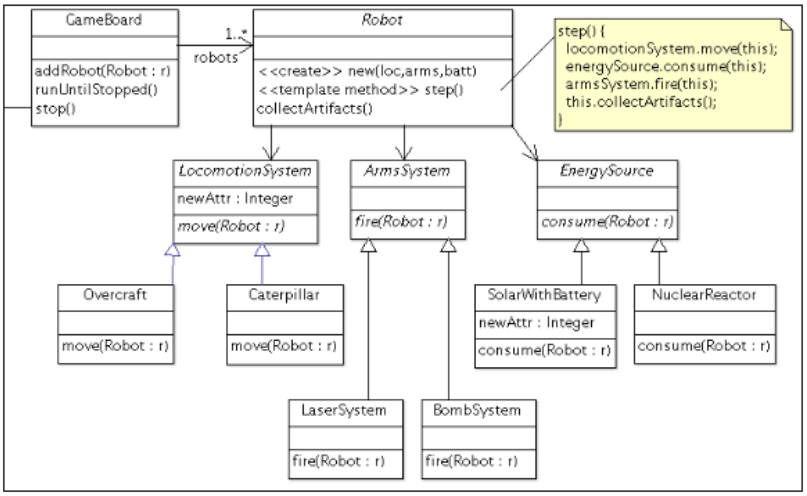
Por lo tanto, n**o es posible** cambiar el sistema de armas en un robot existente sin instanciarlo de nuevo

- ¿Donde almacenaría usted el nivel de carga de la batería? ¿Que implicaría eso si antes de disparar el laser hay que garantizar que la fuente de energía puede satisfacer el consumo del arma?

El nivel de carga se almacenaría como una **Variable de Instancia (V.I.)** en la clase base `Robot` o en la subclase específica, ya que la herencia permite acceso directo al estado y métodos heredados.
Para garantizar que hay energía antes de disparar, la lógica de validación tendría que estar dentro de cada implementación del gancho `fireArms()` en las subclases. Esto resulta poco eficiente porque **duplicarías la lógica de control de energía** en cada variante de robot con láser

Implementado con composición

Esta alternativa profundiza mas la separación de lo que es constante y lo que varía. No solo lo separa en métodos diferentes, sino que también lo encapsula en objetos diferentes.



Ahora:

∗ Un robot tiene tres partes. Un sistema de locomoción, un sistema de armamento, y una fuente de energía.

∗ Hay solo una clase de robot (Robot) que recibe tres parámetros, que corresponden a cada de las partes.

  ∗ Crear un tipo específico de robot implica decidir cuales de las alternativas se utilizará para cada parte.

∗ Los sistemas de locomoción se implementan en la jerarquía de la clase LocomotionSystem. Los sistemas de armas y las fuentes de energía se implementan en las jerarquías ArmsSystem y EnergySource respectivamente.

  ∗ En el ejemplo, pueden obtenerse los 8 tipos de robots posibles combinando esas clases.

∗ Ahora, cuando un robot recibe el mensaje *step(),* sigue utilizando su método plantilla (que garantiza el cumplimiento de los pasos canónicos como lo marca la especificación del framework). Sin embargo el método plantilla ya no envía mensajes al objeto mismo (como lo hacía en el caso de herencia) sino que delega las operaciones en sus partes. Entonces, la plantilla está en el robot, y los ganchos en las partes.

∗ Cuando el robot delega las operaciones gancho en sus partes, se pasa el mismo como parámetro. Esto es necesario porque las partes ya no tienen acceso a las variables de instancia del robot ni de las otras partes.

En consecuencia al implementar los métodos ganchos (en las partes) solo se puede utilizar los métodos que el robot haga públicos. Esto resulta en un escenario mas restrictivo que el que utilizaba herencia.
Además, ahora podemos cambiar el comportamiento del robot en tiempo de ejecución. Alcanza con cambiar la parte afectada por aquella que provee el comportamiento deseado.

- ¿Que debo hacer si aparece una nueva fuente de energía (por ejemplo, ruedas con tracción 4x4)? ¿Cuántas y cuáles clases debo agregar en caso de querer todas las variantes de robots posibles para este nuevo tipo de sistema de locomoción?

Si apareciese un nuevo tipo de locomoción, tracción 4x4 por ejemplo, alcanza con agregar una subclase FourByFourWheels a LocomotionSystems. La nueva clase implementará el método move(Robot : r) como corresponda. Esto es una ventaja sobre el caso visto en la sección anterior (herencia).

- ¿Puedo cambiarle, a un robot existente, el sistema de armas sin tener que instanciar el robot de nuevo?

Si, alcanza con cambiar la parte afectada por aquella que provee el comportamiento deseado.

- ¿Donde almacenaría usted el nivel de carga de la batería? ¿Que implicaría eso si antes de disparar el laser hay que garantizar que la fuente de energía puede satisfacer el consumo del arma?

El nivel de carga se almacenaría directamente en el objeto componente **Battery** (o sus subclases como `NuclearBattery` o `SolarBattery`). El robot ya no "es" un tipo de batería, sino que "**tiene**" una
A diferencia de la herencia, aquí los componentes son objetos separados que colaboran. Cuando el método plantilla `step()` invoca al sistema de armas mediante `armsSystem.fire(this)`, le está pasando una referencia del propio robot (`this`)
El objeto `ArmsSystem` (el láser) recibiría al robot y, a través de su interfaz pública, debería preguntarle al objeto `Battery` si tiene carga suficiente
Como los componentes **no tienen acceso a las variables internas (V.I.)** del robot ni de otros componentes, la comunicación debe ser estrictamente a través de los métodos que el framework provea.

Esto permite que la lógica de "chequeo de energía" sea más genérica y fácil de cambiar, permitiendo incluso cambiar el arma o la batería en tiempo de ejecución simplemente reemplazando el objeto componente.

Discusiones finales sobre Frameworks

※ Como usuarios del framework

Cuando una persona utiliza un framework para construir una aplicación, su experiencia cambia según el mecanismo de extensión

Herencia

› El usuario debe **implementar, extender y redefinir métodos** dentro de la jerarquía de clases del framework

› Tiene la ventaja de poder utilizar las **variables y métodos ya heredados**

› Posee un poder mayor, ya que podría llegar a **cambiar aspectos que el desarrollador original no tuvo en cuenta** y así extender el framework mismo

› El usuario **debe aprender exactamente qué está heredando** y qué puede hacer con ello, además de que queda limitado a la jerarquía del framework (no puede heredar comportamiento de otro lado)

Composición

› El trabajo principal consiste en **instanciar y configurar** objetos, conectando el código propio mediante **callbacks**

› La complejidad disminuye, ya que el usuario solo necesita conocer **algunas clases específicas** y sus mensajes

› Ofrece más libertad estructural: los objetos del usuario son suyos y puede **heredar de donde quiera**

› El usuario **no puede cambiar o extender el framework** y su acceso está limitado únicamente a los objetos que recibe a través de los callbacks

※ Como desarrolladores del framework

Herencia

› El desarrollador deja **ganchos (hooks)** en las clases (tanto abstractas como concretas) y cede el control del flujo mediante **mensajes a self**

› Tiene un mayor grado de predictibilidad: sabe qué esperar del código del usuario si este sigue las reglas definidas

› No necesita pensar todos los hotspots o casos de usos posibles desde el principio

Composición

› Debe crear **objetos configurables** para que el usuario ajuste el comportamiento y pasar el control mediante **callbacks**

› El esfuerzo de diseño es mayor, ya que el desarrollador **debe pensar de antemano todos los hotspots** y sus posibles casos

› Es necesario gestionar explícitamente el estado, asegurándose de **pasar o actualizar toda la información**

› No requiere pasar estados como parámetros, ya que la información reside en las **variables de instancia**

› El desarrollador queda "atado": **no puede cambiar el diseño fácilmente** sin afectar a los usuarios existentes y debe documentar con extrema claridad qué partes se pueden tocar y cuales no

necesaria en cada callback

› La gran ventaja es la **independencia**: el desarrollador no sabe nada del código del usuario, lo que le permite **cambiar el diseño interno del framework** sin preocuparse demasiado por romper las aplicaciones de los usuario, ni necesita explicar (mucho) como funciona dicho framework